

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



≡ Graph API > Graph API overview

LangGraph APIs > Graph API

Graph API overview

📄 Copy page



Graphs

At its core, LangGraph models agent workflows as graphs. You define the behavior of your agents using three key components:

1. State : A shared data structure that represents the current snapshot of your application. It can be any data type, but is typically defined using a shared state schema.
2. Nodes : Functions that encode the logic of your agents. They receive the current state as input, perform some computation or side-effect, and return an updated state.
3. Edges : Functions that determine which `Node` to execute next based on the current state. They can be conditional branches or fixed transitions.

By composing `Nodes` and `Edges`, you can create complex, looping workflows that evolve the state over time. The real power, though, comes from how LangGraph manages that state.

To emphasize: `Nodes` and `Edges` are nothing more than functions—they can contain an LLM or just good ol' code.

In short: *nodes do the work, edges tell what to do next.*

LangGraph's underlying graph algorithm uses message passing to define a general program. When a `Node` completes its operation, it sends messages along one or more edges to other node(s). These recipient nodes then execute their functions, pass the resulting messages to the next set of nodes, and the process continues. Inspired by Google's Pregel system, the program proceeds in discrete "super-steps."

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



by marking themselves as `inactive`. The graph execution terminates when all nodes are `inactive` and no messages are in transit.

StateGraph

The `StateGraph` class is the main graph class to use. This is parameterized by a user defined `State` object.

Compiling your graph

To build your graph, you first define the `state`, you then add `nodes` and `edges`, and then you compile it. What exactly is compiling your graph and why is it needed?

Compiling is a pretty simple step. It provides a few basic checks on the structure of your graph (no orphaned nodes, etc). It is also where you can specify runtime args like `checkpointer`s and breakpoints. You compile your graph by just calling the `.compile` method:

```
graph = graph_builder.compile(...)
```

⚠ You **MUST** compile your graph before you can use it.

State

The first thing you do when you define a graph is define the `State` of the graph. The `State` consists of the `schema of the graph` as well as `reducer functions` which specify how to apply updates to the state. The schema of the `State` will be the input schema to all `Nodes` and `Edges` in the graph, and can be either a `TypedDict` or a `Pydantic` model. All `Nodes` will emit updates to the `State` which are then applied using the specified `reducer` function.

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →

LangChain Docs

By default, the graph will have the same input and output schemas. If you want to change this, you can also specify explicit input and output schemas directly. This is useful when you have a lot of keys, and some are explicitly for input and others for output. See the [guide](#) for more information.

 The higher-level `create_agent` factory in `langchain` does not support Pydantic state schemas.

Multiple schemas

Typically, all graph nodes communicate with a single schema. This means that they will read and write to the same state channels. But, there are cases where we want more control over this:

- Internal nodes can pass information that is not required in the graph's input / output.
- We may also want to use different input / output schemas for the graph. The output might, for example, only contain a single relevant output key.

It is possible to have nodes write to private state channels inside the graph for internal node communication. We can simply define a private schema, `PrivateState`.

It is also possible to define explicit input and output schemas for a graph. In these cases, we define an "internal" schema that contains *all* keys relevant to graph operations. But, we also define `input` and `output` schemas that are sub-sets of the "internal" schema to constrain the input and output of the graph. See [Define input and output schemas](#) for more detail.

Let's look at an example:

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



```
user_input: str

class OutputState(TypedDict):
    graph_output: str

class OverallState(TypedDict):
    foo: str
    user_input: str
    graph_output: str

class PrivateState(TypedDict):
    bar: str

def node_1(state: InputState) -> OverallState:
    # Write to OverallState
    return {"foo": state["user_input"] + " name"}

def node_2(state: OverallState) -> PrivateState:
    # Read from OverallState, write to PrivateState
    return {"bar": state["foo"] + " is"}

def node_3(state: PrivateState) -> OutputState:
    # Read from PrivateState, write to OutputState
    return {"graph_output": state["bar"] + " Lance"}

builder = StateGraph(OverallState, input_schema=InputState, output_schema=OutputState)
builder.add_node("node_1", node_1)
builder.add_node("node_2", node_2)
builder.add_node("node_3", node_3)
builder.add_edge(START, "node_1")
```

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



View example trace



Open a public LangSmith run for this example.

There are two subtle and important points to note here:

1. We pass `state: InputState` as the input schema to `node_1`. But, we write out to `foo`, a channel in `OverallState`. How can we write out to a state channel that is not included in the input schema? This is because a node *can write to any state channel in the graph state*. The graph state is the union of the state channels defined at initialization, which includes `OverallState` and the filters `InputState` and `OutputState`.
2. We initialize the graph with:

```
StateGraph(  
    OverallState,  
    input_schema=InputState,  
    output_schema=OutputState  
)
```

How can we write to `PrivateState` in `node_2`? How does the graph gain access to this schema if it was not passed in the `StateGraph` initialization?

We can do this because `_nodes` can also declare additional state `channels_` as long as the state schema definition exists. In this case, the `PrivateState` schema is defined, so we can add `bar` as a new state channel in the graph and write to it.

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →

LangChain Docs

by `invoke` but visible while streaming:

```
stream = graph.stream_events({"user_input": "My"}, version="v3")
for snapshot in stream.values:
    print(snapshot)
# {'user_input': 'My'}
# {'foo': 'My name', 'user_input': 'My'}
# {'foo': 'My name', 'user_input': 'My', 'bar': 'My name is'}
# {'foo': 'My name', 'user_input': 'My', 'graph_output': 'My name is Lance'}
```



View example trace



Open a public LangSmith run for this example.

To restrict the streamed values to a specific set of channels (e.g. only the output schema), pass `output_keys` :

```
stream = graph.stream_events(
    {"user_input": "My"},
    version="v3",
    output_keys=["graph_output"],
)
for snapshot in stream.values:
    print(snapshot)
# {'graph_output': 'My name is Lance'}
```

If you only need the channels a node actually produced each step (rather than the full accumulated state), use `stream_mode="updates"` instead.

Reducers

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →

LangChain Docs

Every reducer is a binary function with two positional arguments:

- **Left argument:** The current value already stored in state for that key.
- **Right argument:** The update for that key returned by a node.

When a node returns a partial update, LangGraph calls the reducer for each updated key and saves the return value as the new state value:

```
new_value = reducer(left=current_state[key], right=node_update[key])
```

The left argument always comes from accumulated state. The right argument always comes from the latest node update. The following example names both arguments explicitly:

```
from typing import Annotated

from typing_extensions import TypedDict

def append_strings(left: list[str], right: list[str]) -> list[str]:
    """Combine the existing state value (left) with a node update (right)."""
    return left + right

class State(TypedDict):
    tags: Annotated[list[str], append_strings]
```

Suppose the state is `{"tags": ["draft"]}` and a node returns `{"tags": ["review"]}`. LangGraph calls:

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



left argument and keeps only the right.

Default reducer

The default reducer ignores the left argument and replaces the state value with the right argument. This example shows how to use the default reducer:

```
from typing_extensions import TypedDict

class State(TypedDict):
    foo: int
    bar: list[str]
```

In this example, no reducer functions are specified for any key. Let's assume the input to the graph is:

`{"foo": 1, "bar": ["hi"]}`. Let's then assume the first `Node` returns `{"foo": 2}`. This is treated as an update to the state. Notice that the `Node` does not need to return the whole `State` schema - just an update. After applying this update, the `State` would then be `{"foo": 2, "bar": ["hi"]}`. If the second node returns `{"bar": ["bye"]}` then the `State` would then be `{"foo": 2, "bar": ["bye"]}`.

Custom reducers

A custom reducer combines the left and right arguments instead of replacing the state value, which is useful for accumulating values, such as appending updates to a list. This example shows how to specify a custom reducer:

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →

LangChain Docs

```
class State(TypedDict):  
    foo: int  
    bar: Annotated[list[str], add]
```

In this example, we've used the `Annotated` type to specify a reducer function (`operator.add`) for the second key (`bar`). Note that the first key remains unchanged. Let's assume the input to the graph is `{"foo": 1, "bar": ["hi"]}` . Let's then assume the first `Node` returns `{"foo": 2}` . This is treated as an update to the state. Notice that the `Node` does not need to return the whole `State` schema - just an update. After applying this update, the `State` would then be `{"foo": 2, "bar": ["hi"]}` . If the second node returns `{"bar": ["bye"]}` then the `State` would then be `{"foo": 2, "bar": ["hi", "bye"]}` . Notice here that the `bar` key is updated by adding the two lists together.

Overwrite

 In some cases, you may want to bypass a reducer and directly overwrite a state value. LangGraph provides the `Overwrite` type for this purpose. [Learn how to use `Overwrite` here.](#)

Resetting a reducer field

A common source of confusion with reducers: with a merging reducer, returning an empty value does **not** clear the field. Because the reducer merges the right argument into the left one, an empty update is merged in and previously accumulated values are kept.

This pattern matters for error buffers or retry counters that must be cleared between retry attempts:

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



```
class State(TypedDict):
    errors: Annotated[list[str], add]

# node A returns {"errors": ["bad sql"]}
# node B returns {"errors": []}
# state["errors"] is still ["bad sql"]; the empty list is merged in, not cleared
```

To clear the field while keeping a merging reducer, wrap the update with Overwrite :

```
from operator import add
from typing import Annotated

from langgraph.types import Overwrite
from typing_extensions import TypedDict

class State(TypedDict):
    errors: Annotated[list[str], add]

def clear_errors(state: State):
    # Bypass the merging reducer and clear the field
    return {"errors": Overwrite([])}
```

For more information, see Bypass reducers with Overwrite.

Untracked values

`UntrackedValue` is used for state fields that should exist during graph execution but should **never be checkpointed**. When a graph resumes from a checkpoint, untracked values will be reset to their initial state (or be unavailable).

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →

LangChain Docs

- **runtime-only configuration** that should be passed fresh each time

```
import { StateSchema, UntrackedValue, MessagesValue } from "@langchain/langgraph";
import { z } from "zod/v4";

const State = new StateSchema({
  messages: MessagesValue,

  // Untracked: throws if multiple nodes write in same step (guard: true is default)
  dbConnection: new UntrackedValue<DatabaseConnection>(),

  // Untracked with guard: false allows multiple writes, keeps last value
  tempCache: new UntrackedValue(
    z.record(z.string(), z.unknown()),
    { guard: false }
  ),

  // Untracked without a schema (for maximum flexibility)
  runtimeConfig: new UntrackedValue(),
});
```

Behavior:

- During execution: Values are stored and accessible like normal state
- On checkpoint: Untracked values are **excluded** from the checkpoint data
- On resume: Untracked values start fresh (empty or with their default value)
- With `guard: true` (default): Throws error if multiple nodes write in the same step
- With `guard: false`: Multiple writes allowed, last value wins

⚠ Don't use `UntrackedValue` for data you need to persist across interrupts or time travel. Use regular state fields or `ReducedValue` for persistent data.

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



Use `GraphNode` to type node functions defined outside the graph builder:

```
import { GraphNode, StateSchema, Command } from "@langchain/langgraph";
import { z } from "zod/v4";

const State = new StateSchema({
  count: z.number().default(0),
  result: z.string(),
});

// Basic node - receives state, returns partial update
const incrementNode: GraphNode<typeof State> = (state) => {
  return { count: state.count + 1 };
};

// Async node
const fetchNode: GraphNode<typeof State> = async (state, config) => {
  const response = await fetch(`/api/data/${state.count}`);
  return { result: await response.text() };
};

// Node with Command routing - specify valid destinations
const routerNode: GraphNode<{ InputSchema: typeof State; Nodes: "process" | "done" }> = (state) => {
  if (state.count >= 10) {
    return new Command({ goto: "done" });
  }
  return new Command({
    update: { count: state.count + 1 },
    goto: "process"
  });
};
```

State.Node shorthand

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →

LangChain Docs

```
});
```

```
// These are equivalent:
```

```
const myNode1: GraphNode<typeof State> = (state) => ({ step: "done" });
const myNode2: typeof State.Node = (state) => ({ step: "done" });
```

ConditionalEdgeRouter

Use `ConditionalEdgeRouter` for routing functions in conditional edges (no state updates, just routing):

```
import { ConditionalEdgeRouter, END } from "@langchain/langgraph";

const State = new StateSchema({
  shouldContinue: z.boolean(),
  step: z.string(),
});

// Router returns node name(s) or END
const router: ConditionalEdgeRouter<{ InputSchema: typeof State; Nodes: "process" | "summarize" }> = {
  if (!state.shouldContinue) {
    return END;
  }
  return state.step === "initial" ? "process" : "summarize";
};

// Use in graph
graph.addConditionalEdges("check", router);
```

StateSchema.State and StateSchema.Update

Extract the state and update types from a schema for use in custom type definitions:

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



```
// Extract the full state type
type MyState = typeof MyStateSchema.State;
// { messages: BaseMessage[], count: number }

// Extract the update type (partial, with reducer input types)
type MyUpdate = typeof MyStateSchema.Update;
// { messages?: Messages, count?: number }
```

...

Working with messages in graph state

Why use messages?

Most modern LLM providers have a chat model interface that accepts a list of messages as input. LangChain's [chat model interface](#) in particular accepts a list of message objects as inputs. These messages come in a variety of forms such as [HumanMessage](#) (user input) or [AIMessage](#) (LLM response).

To read more about what message objects are, please refer to the [Messages conceptual guide](#).

Using messages in your graph

In many cases, it is helpful to store prior conversation history as a list of messages in your graph state. To do so, we can add a key (channel) to the graph state that stores a list of `Message` objects and annotate it with a reducer function (see `messages` key in the example below). The reducer function is vital to telling the graph how to update the list of `Message` objects in the state with each state update (for example, when a node sends an update). If you don't specify a reducer, every state update will overwrite the list of messages with the most recently provided value. If you wanted to simply append messages to the existing list, you could use `operator.add` as a reducer.

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



existing list, but it will also handle the updates for existing messages correctly.

Serialization

In addition to keeping track of message IDs, the `add_messages` function will also try to deserialize messages into LangChain `Message` objects whenever a state update is received on the `messages` channel.

For more information, see [LangChain serialization/deserialization](#). This allows sending graph inputs / state updates in the following format:

```
# this is supported
{"messages": [HumanMessage(content="message")]}

# and this is also supported
{"messages": [{"type": "human", "content": "message"}]}
```

Since the state updates are always deserialized into LangChain `Messages` when using `add_messages`, you should use dot notation to access message attributes, like `state["messages"][-1].content`.

Below is an example of a graph that uses `add_messages` as its reducer function.

```
from langchain.messages import AnyMessage
from langgraph.graph.message import add_messages
from typing import Annotated
from typing_extensions import TypedDict

class GraphState(TypedDict):
    messages: Annotated[list[AnyMessage], add_messages]
```

MessagesState

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



```
from langgraph.graph import MessagesState

class State(MessagesState):
    documents: list[str]
```

Nodes

In LangGraph, nodes are Python functions (either synchronous or asynchronous) that accept the following arguments:

1. `state` —The state of the graph
2. `config` —A RunnableConfig object that contains configuration information like `thread_id` and tracing information like `tags`
3. `runtime` —A `Runtime` object that contains runtime context and other information like `store` , `stream_writer` , `execution_info` , `server_info` , `heartbeat` (for idle timeout refresh), and `control` (for graceful shutdown)

Similar to `NetworkX` , you add these nodes to a graph using the `add_node` method:

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



```
class State(TypedDict):
    input: str
    results: str

@dataclass
class Context:
    user_id: str

builder = StateGraph(State)

def plain_node(state: State):
    return state

def node_with_runtime(state: State, runtime: Runtime[Context]):
    print("In node: ", runtime.context.user_id)
    return {"results": f"Hello, {state['input']}!"}

def node_with_execution_info(state: State, runtime: Runtime):
    print("In node with thread_id: ", runtime.execution_info.thread_id)
    return {"results": f"Hello, {state['input']}!"}

builder.add_node("plain_node", plain_node)
builder.add_node("node_with_runtime", node_with_runtime)
builder.add_node("node_with_execution_info", node_with_execution_info)
...
```

Behind the scenes, functions are converted to `RunnableLambda`, which add batch and async support to your function, along with native tracing and debugging.

If you add a node to a graph without specifying a name, it will be given a default name equivalent to the function name.

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →

LangChain Docs

When you compile with a checkpoint, LangGraph saves checkpoints at super step boundaries, not mid-function inside a node. If execution stops and later resumes (for example after an interrupt or a retry), the affected **node** runs again from the start of its function. Code and side effects before the pause run again.

Idempotency. Design **node** logic so re-execution does not corrupt state. If a node inserts a database row, running it twice should not create duplicate rows unless that is intentional. Use idempotency keys, upserts, or read-before-write checks. For effects around `interrupt()`, see Side effects called before `interrupt` must be idempotent.

Graph changes. Determinism rules about code changes do not apply to graph structure. You can add or remove **nodes** and edges without breaking resume for existing threads. Resumed runs use saved state and execute whatever graph you compile now.

Tasks and interrupts inside a node. If a **node** calls tasks or interrupt, stricter determinism rules apply on resume. LangGraph restores completed **task** results from the checkpoint, but changing **task** or interrupt order in code before the resume point can mismatch cached values. A Functional API **entrypoint** compiles to a single **node** that runs the whole entrypoint method this way. See Determinism, Idempotency, and Using tasks in nodes.

Using tasks in nodes

If a node contains multiple operations, you may find it easier to implement each operation as a task instead of splitting the logic across multiple nodes. Task results are checkpointed when the graph uses a checkpoint, so resuming a thread can skip completed **task** work inside the node.

Original With task

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



```
from typing_extensions import TypedDict

class State(TypedDict):
    url: str
    result: NotRequired[str]

def call_api(state: State):
    """Example node that makes an API request."""
    result = requests.get(state["url"]).text[:100]
    return {"result": result}

builder = StateGraph(State)
builder.add_node("call_api", call_api)
builder.add_edge(START, "call_api")
builder.add_edge("call_api", END)

checkpointer = InMemorySaver()
graph = builder.compile(checkpointer=checkpointer)

thread_id = str(uuid7())
config = {"configurable": {"thread_id": thread_id}}

graph.invoke({"url": "https://www.example.com"}, config)
```

View example trace



Open a public LangSmith run for this example.

START node

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



```
graph.add_edge(START, "node_a")
```

END node

The `END` Node is a special node that represents a terminal node. This node is referenced when you want to denote which edges have no actions after they are done.

```
from langgraph.graph import END

graph.add_edge("node_a", END)
```

Node caching

LangGraph supports caching of tasks/nodes based on the input to the node. To use caching:

- Specify a cache when compiling a graph (or specifying an entrypoint)
- Specify a cache policy for nodes. Each cache policy supports:
 - `key_func` used to generate a cache key based on the input to a node, which defaults to a `hash` of the input with `pickle`.
 - `ttl` , the time to live for the cache in seconds. If not specified, the cache will never expire.

For example:

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →

LangChain Docs

```
class State(TypedDict):
    x: int
    result: int

builder = StateGraph(State)

def expensive_node(state: State) -> dict[str, int]:
    # expensive computation
    time.sleep(2)
    return {"result": state["x"] * 2}

builder.add_node("expensive_node", expensive_node, cache_policy=CachePolicy(ttl=10))
builder.set_entry_point("expensive_node")
builder.set_finish_point("expensive_node")

graph = builder.compile(cache=InMemoryCache())

print(graph.invoke({"x": 5}, stream_mode='updates'))
# [{'expensive_node': {'result': 10}}]

print(graph.invoke({"x": 5}, stream_mode='updates'))
# [{'expensive_node': {'result': 10}, '__metadata__': {'cached': True}}]
```

❗ `set_entry_point(node)` defines the first node the graph will execute. It is equivalent to `builder.add_edge(START, node)`.

`set_finish_point(node)` defines the last node in the graph. It is equivalent to `builder.add_edge(node, END)`.

Both methods are valid but `add_edge(START, ...)` and `add_edge(..., END)` are the recommended modern syntax.

1. First run takes two seconds to run (due to mocked expensive computation).

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →

LangChain Docs

are a few key types of edges:

- Normal Edges: Go directly from one node to the next.
- Conditional Edges: Call a function to determine which node(s) to go to next.
- Entry Point: Which node to call first when user input arrives.
- Conditional Entry Point: Call a function to determine which node(s) to call first when user input arrives.

A node can have multiple outgoing edges. If a node has multiple outgoing edges, **all** of those destination nodes will be executed in parallel as a part of the next superstep.

⚠ For each node, choose one routing mechanism: use normal edges for static routing, or use conditional edges / Command for dynamic routing. Do not mix normal edges and dynamic routing from the same node, because both paths can execute and make graph behavior harder to reason about.

Normal edges

If you **always** want to go from node A to node B, you can use the add_edge method directly.

```
graph.add_edge("node_a", "node_b")
```

Conditional edges

If you want to **optionally** route to one or more edges (or optionally terminate), you can use the add_conditional_edges method. This method accepts the name of a node and a "routing function" to call after that node is executed:

```
graph.add_conditional_edges("node_a", routing_function)
```

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →

LangChain Docs

You can optionally provide a dictionary that maps the `routing_function` 's output to the name of the next node.

```
graph.add_conditional_edges("node_a", routing_function, {True: "node_b", False:
```



Use `Command` instead of conditional edges if you want to combine state updates and routing in a single function.

Entry point

The entry point is the first node(s) that are run when the graph starts. You can use the `add_edge` method from the virtual `START` node to the first node to execute to specify where to enter the graph.

```
from langgraph.graph import START

graph.add_edge(START, "node_a")
```

Conditional entry point

A conditional entry point lets you start at different nodes depending on custom logic. You can use `add_conditional_edges` from the virtual `START` node to accomplish this.

```
from langgraph.graph import START

graph.add_conditional_edges(START, routing_function)
```

You can optionally provide a dictionary that maps the `routing_function` 's output to the name of the next node.

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →

LangChain Docs

shared state. However, there can be cases where the exact edges are not known ahead of time and/or you may want different versions of `State` to exist at the same time. A common example of this is with map-reduce design patterns. In this design pattern, a first node may generate a list of objects, and you may want to apply some other node to all those objects. The number of objects may be unknown ahead of time (meaning the number of edges may not be known) and the input `State` to the downstream `Node` should be different (one for each generated object).

To support this design pattern, LangGraph supports returning `Send` objects from conditional edges. `Send` takes two arguments: first is the name of the node, and second is the state to pass to that node.

```
from langgraph.types import Send

def continue_to_jokes(state: OverallState):
    return [Send("generate_joke", {"subject": s}) for s in state['subjects']]

graph.add_conditional_edges("node_a", continue_to_jokes)
```

Command

`Command` is a versatile primitive for controlling graph execution. It accepts four parameters:

- `update` : Apply state updates (similar to returning updates from a node).
- `goto` : Navigate to specific nodes (similar to conditional edges).
- `graph` : Target a parent graph when navigating from subgraphs.
- `resume` : Provide a value to resume execution after an interrupt.

`Command` is used in three contexts:

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



Return from nodes

update **and** goto

Return `Command` from node functions to update state and route to the next node in a single step:

```
def my_node(state: State) -> Command[Literal["my_other_node"]]:
    return Command(
        # state update
        update={"foo": "bar"},
        # control flow
        goto="my_other_node"
    )
```

With `Command` you can also achieve dynamic control flow behavior (identical to conditional edges):

```
def my_node(state: State) -> Command[Literal["my_other_node"]]:
    if state["foo"] == "bar":
        return Command(update={"foo": "baz"}, goto="my_other_node")
```

Use `Command` when you need to **both** update state **and** route to a different node. If you only need to route without updating state, use conditional edges instead.

- ❗ When returning `Command` in your node functions, you must add return type annotations with the list of node names the node is routing to, e.g.
`Command[Literal["my_other_node"]]`. This is necessary for the graph rendering and tells LangGraph that `my_node` can navigate to `my_other_node`.

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



graph

If you are using [subgraphs](#), you can navigate from a node within a subgraph to a different node in the parent graph by specifying `graph=Command.PARENT` in [Command](#) :

```
def my_node(state: State) -> Command[Literal["other_subgraph"]]:
    return Command(
        update={"foo": "bar"},
        goto="other_subgraph", # where `other_subgraph` is a node in the parent
        graph=Command.PARENT
    )
```

❗ Setting `graph` to `Command.PARENT` will navigate to the closest parent graph.

When you send updates from a subgraph node to a parent graph node for a key that's shared by both parent and subgraph [state schemas](#), you **must** define a [reducer](#) for the key you're updating in the parent graph state. See this [example](#).

This is particularly useful when implementing [multi-agent handoffs](#). Check out [Navigate to a node in a parent graph](#) for detail.

Input to invoke or stream

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



```
# WRONG - graph resumes from the latest checkpoint
# (last step that ran), appears stuck
- graph.invoke(Command(update={
-     "messages": [{"role": "user", "content": "follow up"}]
- }), config)

# CORRECT - plain dict restarts from __start__
+ graph.invoke( {
+     "messages": [{"role": "user", "content": "follow up"}]
+ }, config)
```

resume

Use `Command(resume=...)` to provide a value and resume graph execution after an interrupt. The value passed to `resume` becomes the return value of the `interrupt()` call inside the paused node:

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



```
class State(TypedDict):
    messages: list[dict]

def human_review(state: State):
    # Pauses the graph and waits for a value
    answer = interrupt("Do you approve?")
    return {"messages": [{"role": "user", "content": answer}]}

graph = (
    StateGraph(State)
    .add_node("human_review", human_review)
    .add_edge(START, "human_review")
    .add_edge("human_review", END)
    .compile(checkpointer=InMemorySaver())
)

config = {"configurable": {"thread_id": "graph-api-resume"}}

# First run - hits the interrupt and pauses
stream = graph.stream_events({"messages": []}, config, version="v3")
_ = stream.output # drive the stream to completion
print(stream.interrupts)

# Resume with a value - the interrupt() call returns "yes"
resumed = graph.stream_events(Command(resume="yes"), config, version="v3")
final = resumed.output
```

View example trace



Open a public LangSmith run for this example.

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →

LangChain Docs

to modify state (e.g., saving customer information looked up during a conversation) and `goto` to route to a specific node after the tool completes.

⚠ When used inside tools, `goto` adds a dynamic edge—any static edges already defined on the node that called the tool will still execute. For each node, use either tool-driven dynamic routing or static edges to route to the next nodes, not both.

Refer to [Use inside tools](#) for detail.

Graph migrations

LangGraph can easily handle migrations of graph definitions (nodes, edges, and state) even when using a checkpointer to track state.

- For threads at the end of the graph (i.e. not interrupted) you can change the entire topology of the graph (i.e. all nodes and edges, remove, add, rename, etc)
- For threads currently interrupted, we support all topology changes other than renaming / removing nodes (as that thread could now be about to enter a node that no longer exists) — if this is a blocker please reach out and we can prioritize a solution.
- For modifying state, we have full backwards and forwards compatibility for adding and removing keys
- State keys that are renamed lose their saved state in existing threads
- State keys whose types change in incompatible ways could currently cause issues in threads with state from before the change — if this is a blocker please reach out and we can prioritize a solution.

💡 For changes that are technically compatible but alter business logic, such as rewriting the tool set or restructuring conversation flow, see [Business compatibility](#). That page covers pinning a behavioral version in state so existing threads keep the old path while new threads pick up the latest version.

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →

LangChain Docs

```
@dataclass
class ContextSchema:
    llm_provider: str = "openai"

graph = StateGraph(State, context_schema=ContextSchema)
```

You can then pass this context into the graph using the `context` parameter of the `invoke` method.

```
graph.invoke(inputs, context={"llm_provider": "anthropic"})
```

You can then access and use this context inside a node or conditional edge:

```
from langgraph.runtime import Runtime

def node_a(state: State, runtime: Runtime[ContextSchema]):
    llm = get_llm(runtime.context.llm_provider)
    # ...
```

See [Add runtime configuration](#) for a full breakdown on configuration.

Recursion limit

The recursion limit sets the maximum number of super-steps the graph can execute during a single execution. Once the limit is reached, LangGraph will raise

`GraphRecursionError`. Starting in version 1.0.6, the default recursion limit is set to 1000 steps. The recursion limit can be set on any graph at runtime, and is passed to `invoke` / `stream` via the config dictionary. Importantly, `recursion_limit` is a

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



Read [Recursion limit](#) to learn more about how the recursion limit works.

Accessing and handling the recursion counter

The current step counter is accessible in `config["metadata"]["langgraph_step"]` within any node, allowing for proactive recursion handling before hitting the recursion limit. This enables you to implement graceful degradation strategies within your graph logic.

How it works

The step counter is stored in `config["metadata"]["langgraph_step"]`. `LangGraph` increments this counter as the graph executes and raises a `GraphRecursionError` once the configured `recursion_limit` is exceeded.

Accessing the current step counter

You can access the current step counter within any node to monitor execution progress.

```
from langchain_core.runnables import RunnableConfig
from langgraph.graph import StateGraph

def my_node(state: dict, config: RunnableConfig) -> dict:
    current_step = config["metadata"]["langgraph_step"]
    print(f"Currently on step: {current_step}")
    return state
```

Proactive recursion handling

`LangGraph` provides a `RemainingSteps` managed value that tracks how many steps remain before hitting the recursion limit. This allows for graceful degradation within your graph.

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



```
remaining_steps: RemainingSteps  # Managed value - tracks steps until limit

def reasoning_node(state: State) -> dict:
    # RemainingSteps is automatically populated by LangGraph
    remaining = state["remaining_steps"]

    # Check if we're running low on steps
    if remaining <= 2:
        return {"messages": ["Approaching limit, wrapping up..."]}

    # Normal processing
    return {"messages": ["thinking..."]}

def route_decision(state: State) -> Literal["reasoning_node", "fallback_node"]:
    """Route based on remaining steps"""
    if state["remaining_steps"] <= 2:
        return "fallback_node"
    return "reasoning_node"

def fallback_node(state: State) -> dict:
    """Handle cases where recursion limit is approaching"""
    return {"messages": ["Reached complexity limit, providing best effort answer"]}

# Build graph
builder = StateGraph(State)
builder.add_node("reasoning_node", reasoning_node)
builder.add_node("fallback_node", fallback_node)
builder.add_edge(START, "reasoning_node")
builder.add_conditional_edges("reasoning_node", route_decision)
builder.add_edge("fallback_node", END)

graph = builder.compile()

# RemainingSteps works with any recursion_limit
result = graph.invoke({"messages": []}, {"recursion_limit": 10})
```

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →

LangChain Docs

```
messages: Annotated[List, lambda x, y: x + y]
remaining_steps: RemainingSteps

# Proactive Approach (recommended) - using RemainingSteps
def agent_with_monitoring(state: State) -> dict:
    """Proactively monitor and handle recursion within the graph"""
    remaining = state["remaining_steps"]

    # Early detection - route to internal handling
    if remaining <= 2:
        return {
            "messages": ["Approaching limit, returning partial result"]
        }

    # Normal processing
    return {"messages": [f"Processing... ({remaining} steps remaining)"]}

def route_decision(state: State) -> Literal["agent", END]:
    if state["remaining_steps"] <= 2:
        return END
    return "agent"

# Build graph
builder = StateGraph(State)
builder.add_node("agent", agent_with_monitoring)
builder.add_edge(START, "agent")
builder.add_conditional_edges("agent", route_decision)
graph = builder.compile()

# Proactive: Graph completes gracefully
result = graph.invoke({"messages": []}, {"recursion_limit": 10})

# Reactive Approach (fallback) - catching error externally
try:
    result = graph.invoke({"messages": []}, {"recursion_limit": 10})
except GraphRecursionError as e:
```

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



Approach	Detection	Handling	Control Flow
Proactive (using <code>RemainingSteps</code>)	Before limit reached	Inside graph via conditional routing	Graph continues to completion node
Reactive (catching <code>GraphRecursionError</code>)	After limit exceeded	Outside graph in try/catch	Graph execution terminated

Proactive advantages:

- Graceful degradation within the graph
- Can save intermediate state in checkpoints
- Better user experience with partial results
- Graph completes normally (no exception)

Reactive advantages:

- Simpler implementation
- No need to modify graph logic
- Centralized error handling

Other available metadata

Along with `langgraph_step` , the following metadata is also available in `config["metadata"]` :

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



```
print(f"Path: {metadata['langgraph_path']}")  
print(f"Checkpoint NS: {metadata['langgraph_checkpoint_ns']}")  
  
return state
```

Visualization

It's often nice to be able to visualize graphs, especially as they get more complex.

LangGraph comes with several built-in ways to visualize graphs. See [Visualize your graph](#) for more info.

Observability and Tracing

To trace, debug and evaluate your agents, use [LangSmith](#).

Learn more

- [How to use the Graph API](#)
- [Functional API conceptual overview](#)
- [Choosing between Graph API and Functional API](#)

 [Connect these docs](#) to Claude, VSCode, and more via MCP for real-time answers.

 [Edit this page on GitHub](#) or [file an issue](#).

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



LangChain Docs

Resources

[Forum](#)

[Changelog](#)

[LangChain Academy](#)

[Contact Sales](#)

Company

[Home](#)

[Trust Center](#)

[Careers](#)

[Blog](#)